

Presentation 3

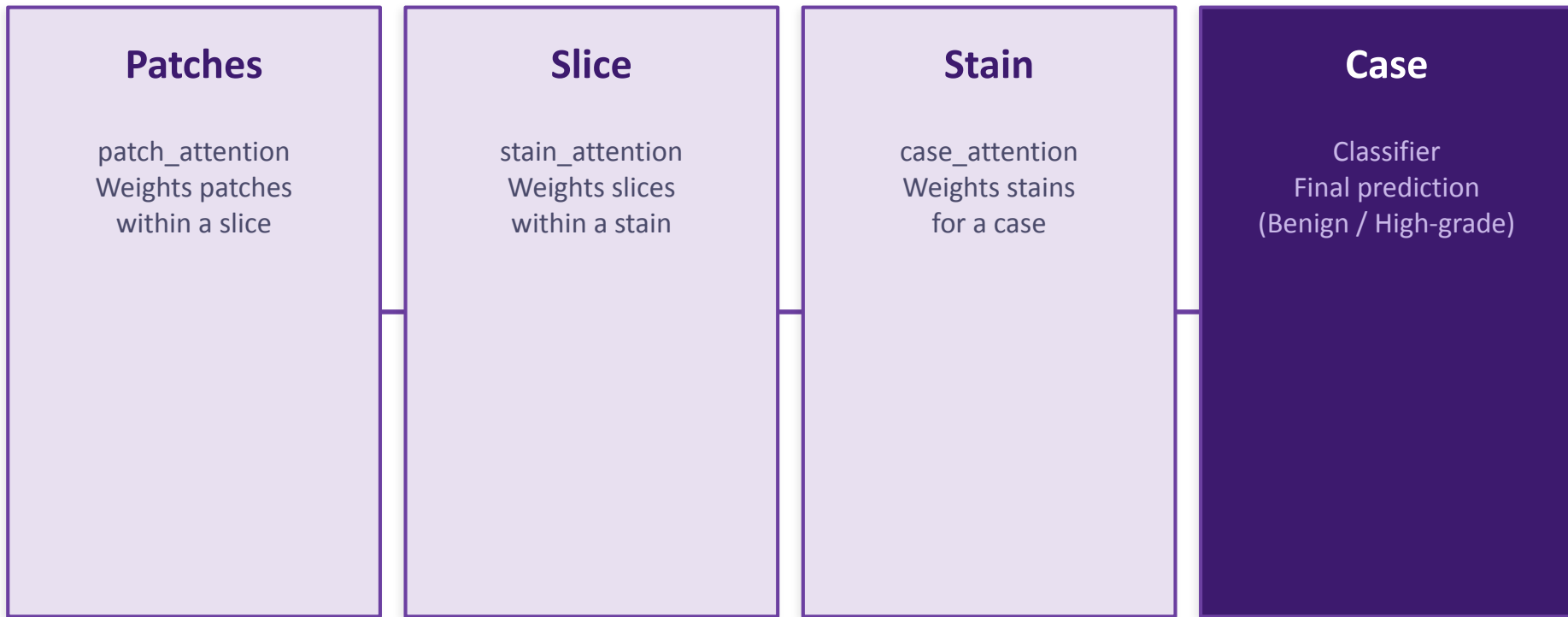
STAT 390 | Winter 2026 | Team 6

Northwestern

Mathlete

Abhi Vinnakota

Attention Hierarchy: Bottom-up



Each attention module answers a different question at a different level of the hierarchy

Part a

Argument: "Peaky patch attention → strong class evidence → slice should receive high slice attention → slice attention is redundant."

Different Questions

patch_attention asks: which patches within this slice matter?

stain_attention asks: which slices within this stain matter?

These are fundamentally separate questions. One cannot substitute for the other.

Relative vs. Absolute

Even if all 4 H&E slices have peaky patch attention, stain_attention must still determine which of the 4 slices is most diagnostically relevant relative to the others.

This relative comparison is hard to determine from patch attention alone.

Part b

What happens if we remove slice-level attention?

I. Uniform Slice Attention

All slices contribute equally.
stain_attention (slice level) is essentially averaging. Removing it has minimal impact. Reducing signal loss without losing useful structure.

II. Peak Slice Attention

One slice clearly dominates.
stain_attention (slice level) is doing real work by focusing on it. Removing it forces equal contribution from all slices, diluting the key signal.

III. Mixed (Realistic)

Some cases uniform, some peaky.
Removing slice attention hurts peaky cases while barely affecting uniform ones. We can't know in advance which regime applies.

Recommendation: Keep Slice-Level Attention

Answers a Different Question

patch_attention and stain_attention solve distinct aggregation problems. Peak patch weights tell you nothing about a slice's importance relative to sibling slices.

Scenario III is Most Realistic

The data contains a mix of uniform and peaky cases across patients. Slice attention can learn to be approximately uniform when needed but selective when it matters.

Cost of Removal > Benefit

Removing slice attention saves little computation but risks degrading performance on the hard peaky cases, precisely those most clinically relevant.

Main Question

Should we allow variable sizes or keep a fixed size of 224×224 ?

This affects three things:

1. Computational cost (FLOPs)
2. Detection of small regions
3. Model underfitting/overfitting

Current Setup

Patch size 224×224

Feature extractor KimiaNet

Stored embeddings 4096-dim pooled features

Projection 4096 $\rightarrow$ 512-dim

Computational Complexity

FLOPs per patch $\propto H^2 \times W^2$ (quadratic in image dimensions)

4×

Doubling resolution

more FLOPs per patch

Once

Embeddings precomputed

changing size means full recomputation

80K

Patches to reprocess

changing size means full recomputation

Detection of Small Pathological Regions

Case for Higher Resolution

More pixels per patch

More detail could help capture subtle differences in cell shape and size

Smaller features

Very small regions of interest might get lost when the image is compressed into a feature vector

Case for Current Resolution

Patches already frame the tissue consistently

Each patch is cut to follow the tissue boundary. What's inside each patch is already standardized regardless of resolution

The question isn't about pixels

Classification depends on where melanocytes appear across the tissue. That's a pattern across many patches, not something more pixels per patch can help with

Model Accuracy: Bias & Variance

Lower Resolution

- Lower compute
- Loses fine detail
- Underfits pathology features
- High bias risk as a result

Current Resolution

- KimiaNet native size
- ImageNet normalization aligned
- Embeddings already stored
- Balanced bias/variance

Higher Resolution

- More FLOPs
- Needs full recompute
- Risk: higher variance on low patient diversity
- Diminishing returns post-pooling

Recommendation: Fix resolution at 224 x 224

01

KimiaNet alignment

KimiaNet was pre trained at 224x224 on pathology images. Using any other size puts the encoder out of distribution.

02

Embeddings are precomputed

All 4096-dim embeddings are already stored (config.py). Changing resolution requires recomputing everything (quite expensive).

03

Patching already controls scale

Patches are cut to follow the tissue boundary, so the content inside each patch is already geometrically consistent. Higher resolution adds more pixels of the same tissue, not more tissue.

Programming

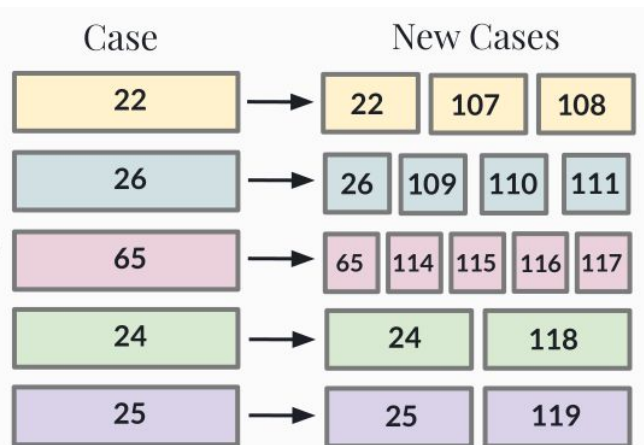
Henry Chau

New Base Code [↗](#)

- Used base code copied from
`/projects/e32998/STAT390_Krish/Code/Code4_reduce_runtime`
- Replaced attention analysis code with the updated scripts from
`/projects/e32998/STAT390_Krish/Code/Code6_attention_check_v2/*`
- Integrated changes from Vicky's implementation of task 5 and 6 from last presentation to meet mandatory tasks 1–3
 - Removed dropout
 - Use gated attention
 - Add entropy regularization
- All results use the code with mandatory changes included and new attention analysis code (seems to be important since other teams reported it affecting results)

Critique of previous case splitting logic

- The former logic in `make_splits.py` specified the newly created benign cases and prevents them from being assigned to the testing set
- Issue 1: Preventing modified cases from being included in the testing set ignores the possibility that there are problems with predictions on them and goes against optional task 2 to test all benign cases
- Issue 2: Data leakage between training and validation set inflate validation performance



- Krish's split train benign cases: [2, 4, 22, 23, 24, 25, 30, 36, 43, 45, 56, 78, 82, 85, 93, 94, 108, 109, 110, 118, 119]
- Krish's split validation benign cases: [26, 27, 46, 53, 90, 107, 111]
- Log file of Krish's data splits [↗](#), detection of leakage [↗](#)

Better data leakage detection in data_utils.py ↗

- The former logic in data_utils.py which make_splits.py depends on to detect data leakage cannot detect data leakage between the newly created cases as I stated in my previous presentation
- I have applied a fix by loading in a list of grouped cases from config and replacing cases with original case id so checking the sets works, pseudocode below

```
case_remap = {apparent_case:original_case for group in CASES for apparent_case in group}
*_case = {case_remap[apparent_case] for apparent_case in *_cases}
```

- Currently have to manually update grouped cases in config.py as the new case ids based on Team5's case splits does not have an intuitive mapping
- Case 82 was not split in Team5's version and I kept it as is since it has only 6 h&e slices and allowing to be over 5 slices avoids the issue of no melan and sox10 slice

Coder Task 2 Implementation (splits) [↗](#)

- The exclusion of created benign cases in `make_splits.py` is counter-productive evaluating all benign cases so I made a significant rewrite to this script
- I needed to make splits such that every case is being tested and there is no data leakage
- I decided on deterministic `StratifiedGroupKFold` from `sklearn` to handle the data splitting process and offload the complex logic involved

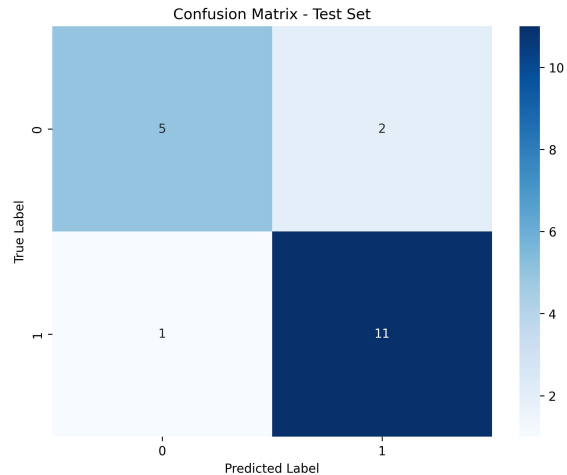
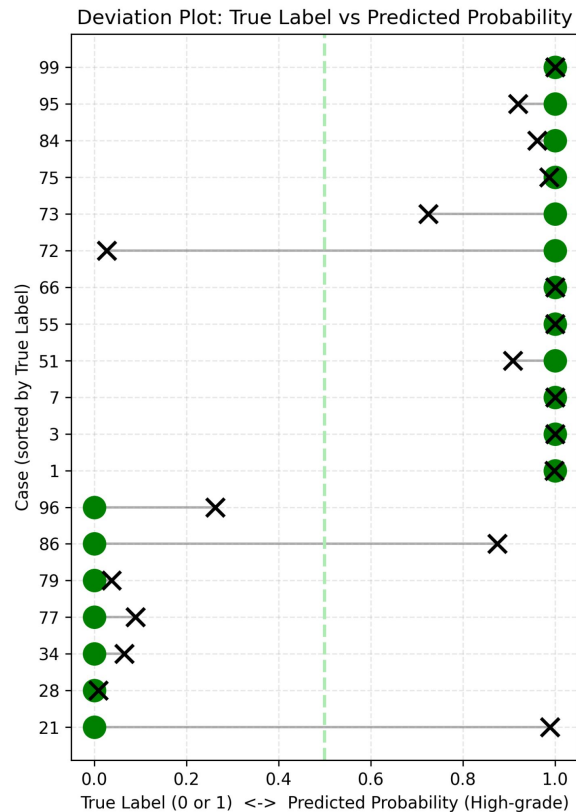
```
denom = simplify_split_ratios([train_ratio, val...]) # find optimal simple denominator
groups = np.array(case_ids).replace_apparent_id_with_real()
sgkf = StratifiedGroupKFold(n_splits=sum(denom)) # for our data, a 5-way split
splits = [test_idx for _, test_idx in sgkf.split(..., groups)] # split on real case id
for i in range(times_needed_to_test_all_cases):
    train_cases, val_cases, test_cases = assign_splits(splits)
    save_case_splits()
    splits.rotate_by_length_tested() # shifts splits so tested cases are different
```


Coder Task 2 Implementation (splits results) [↗](#)

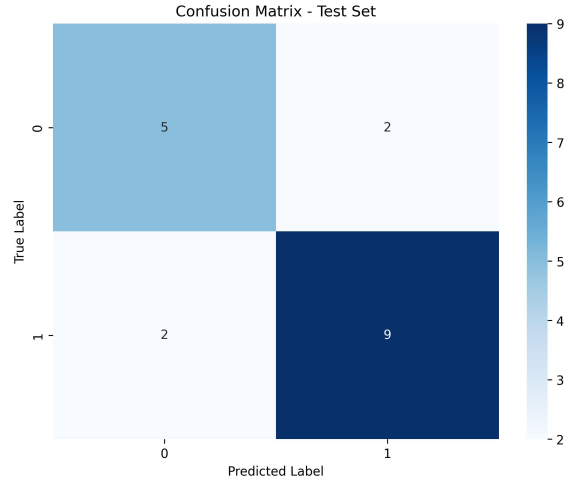
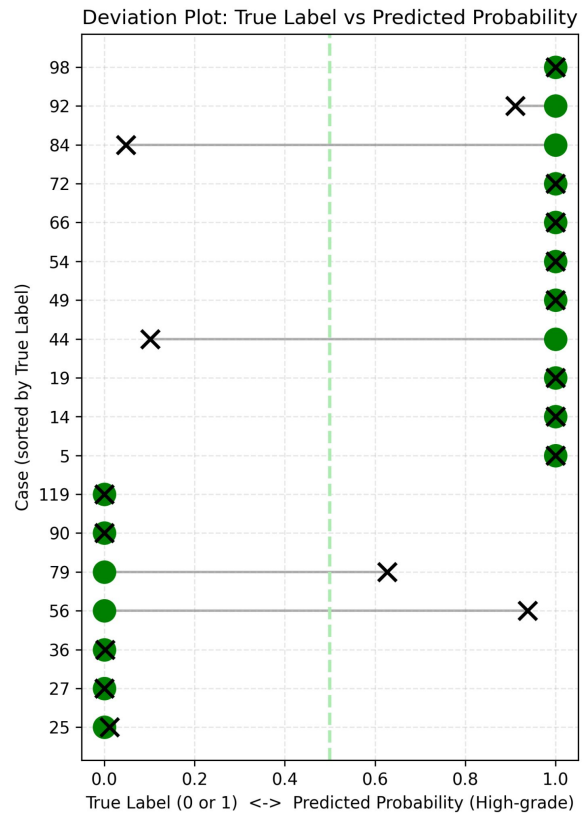
- Created 5 non-overlapping data splits in the manner of the pre-made splits
- No data leakage according to new data leakage code
- Even with grouped restrictions, tight high-grade ratio band (61.11%–63.16%), although when I tested with random shuffle, high-grade ratio band was much worse (~47%–71%)

	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
Split 1	Train	Train	Train	Val	Test
Split 2	Train	Train	Val	Test	Train
Split 3	Train	Val	Test	Train	Train
Split 4	Val	Test	Train	Train	Train
Split 5	Test	Train	Train	Train	Val

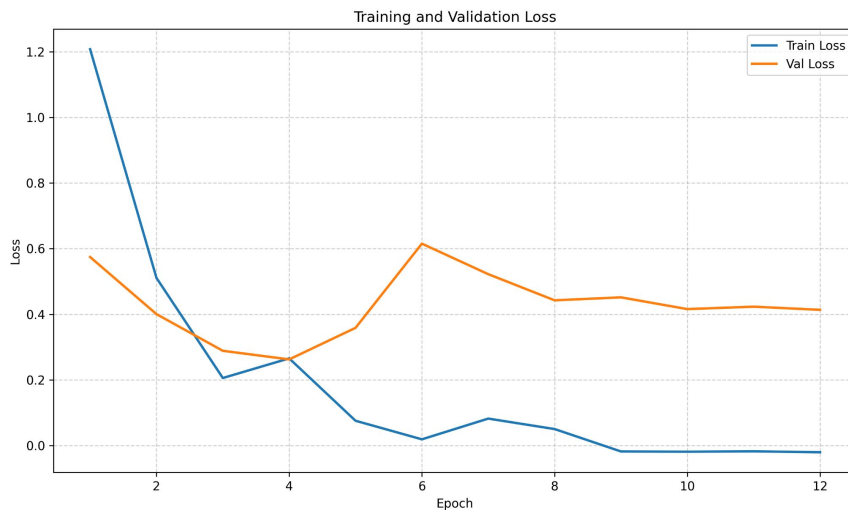
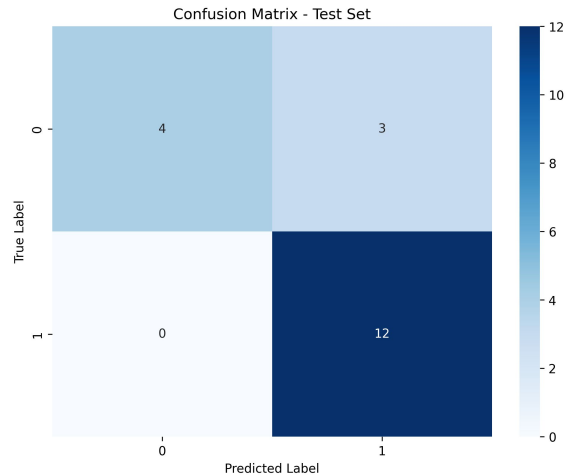
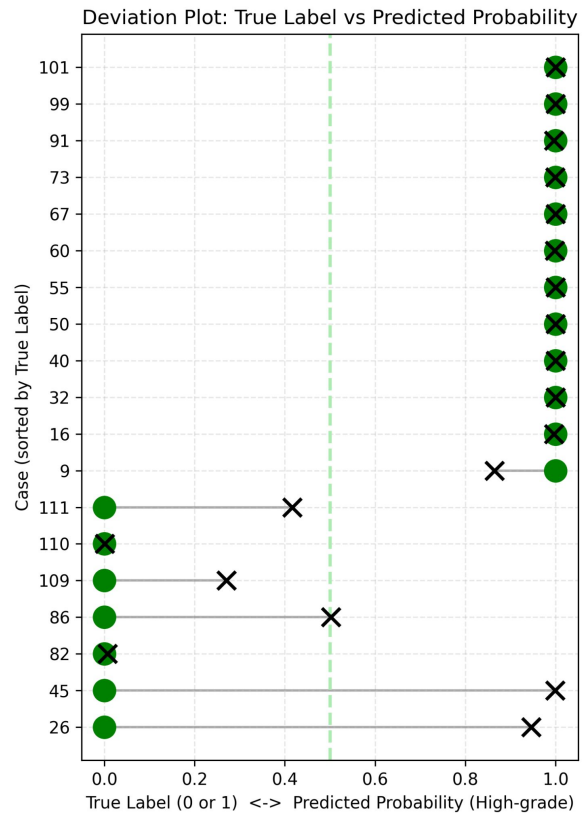
Split Control (Krish's) results [↗](#)



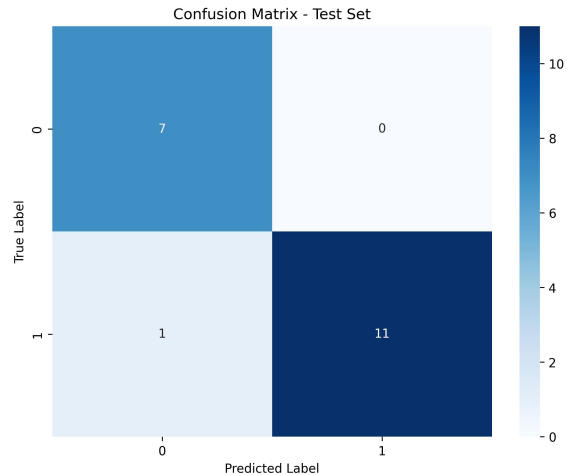
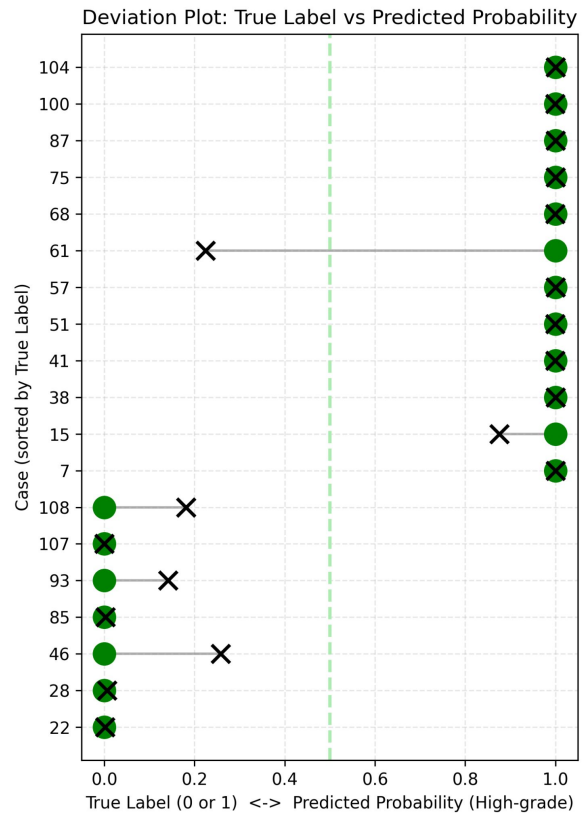
Split 1 results [↗](#)



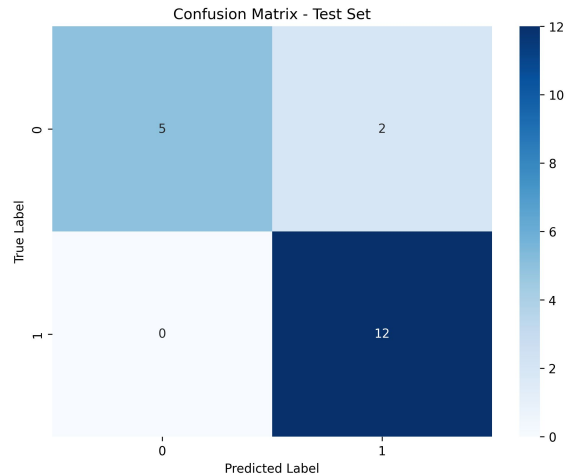
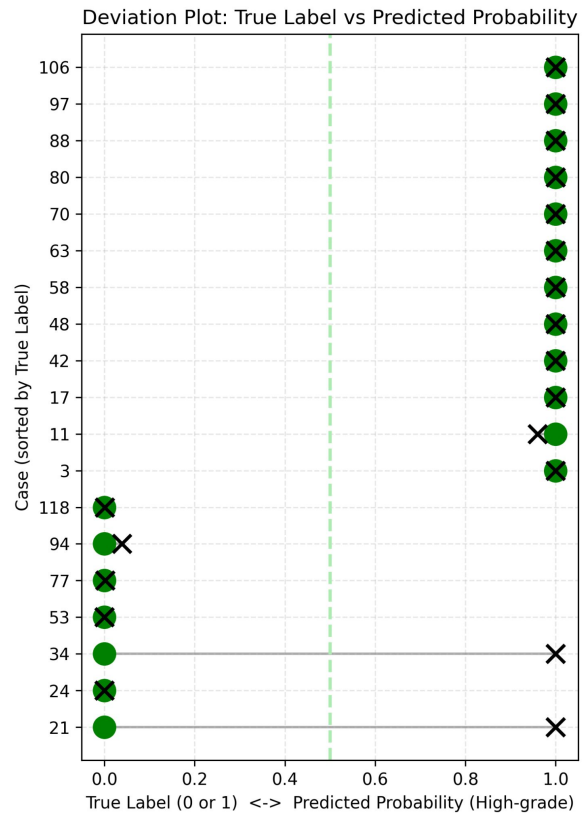
Split 2 results ↗



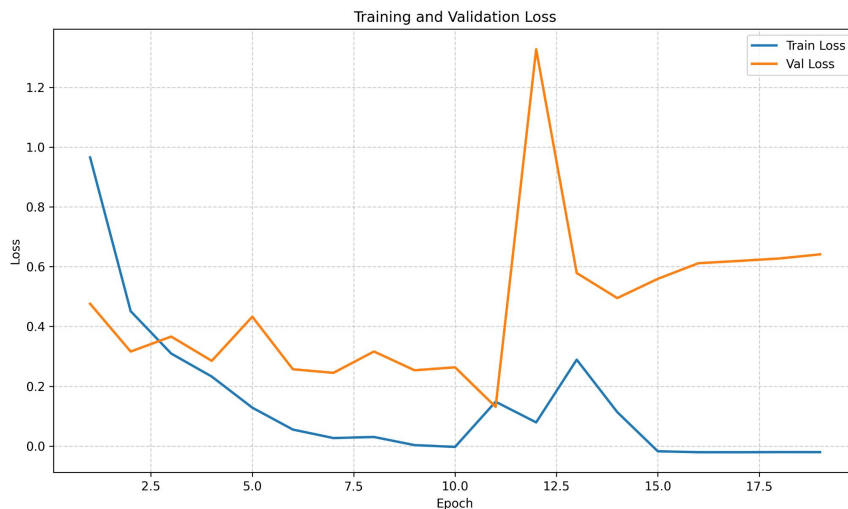
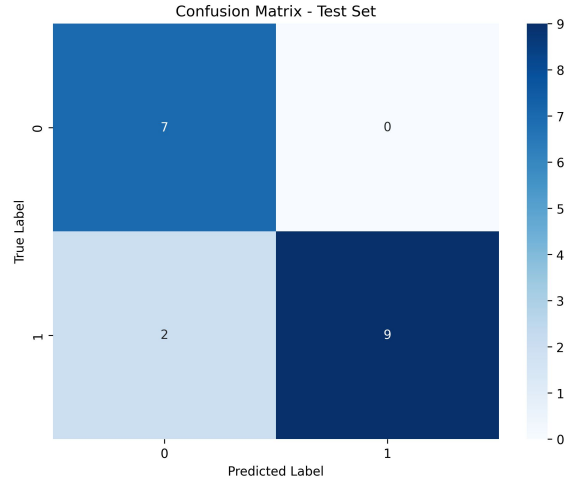
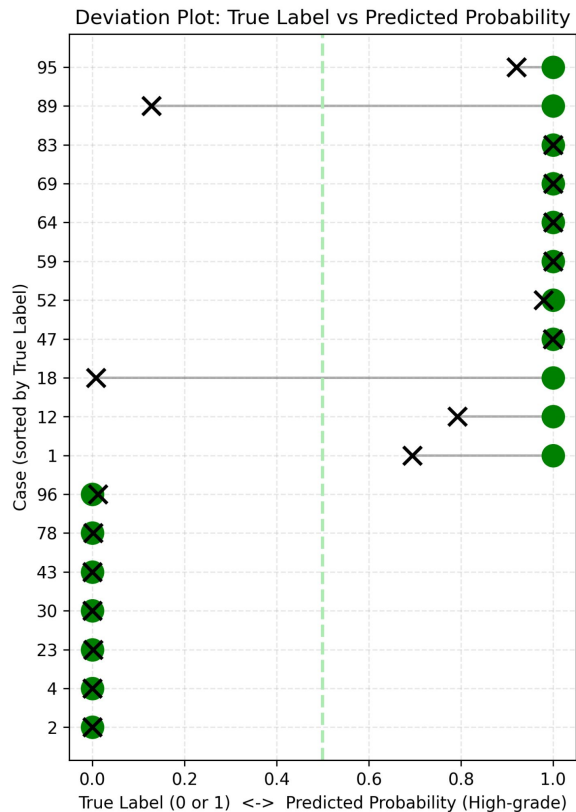
Split 3 results ↗



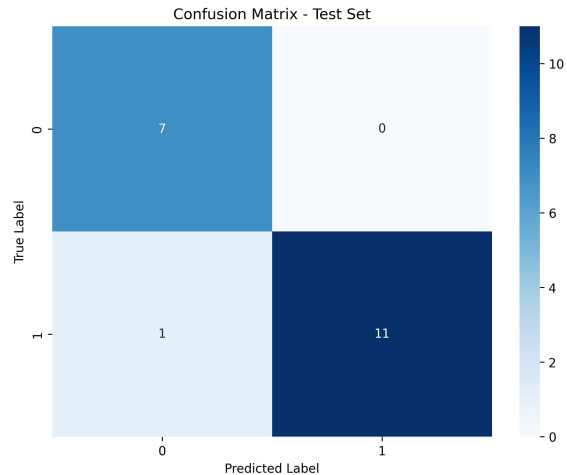
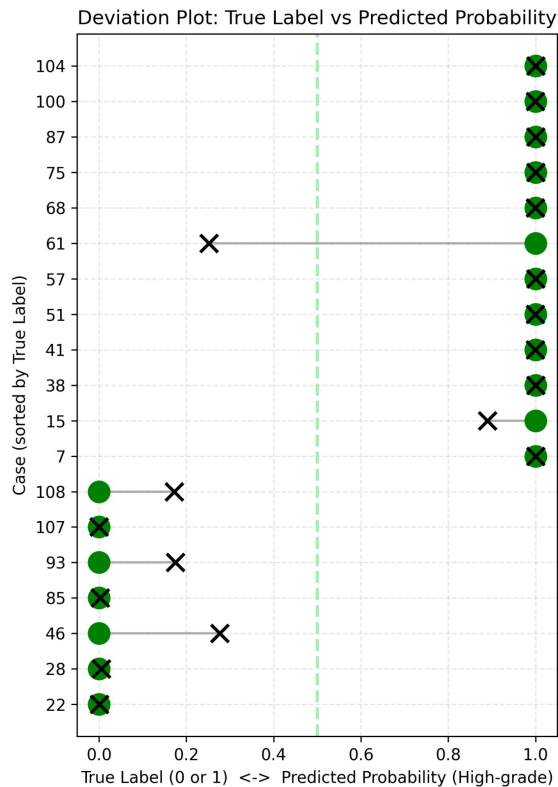
Split 4 results ↗



Split 5 results [↗](#)



Split 3 extended results [↗](#)



Experimental model improvement (Programming)

Vicky Xu

Mandatory Task implementation

4. Use the splits -

/projects/e32998/STAT390_Krish/Code/Code4_reduce_runtime/data_splits/data_splits.npz

5. 30-Epoch Minimum: Guarantees full model convergence and allows sufficient time for the regularization terms to optimize decision boundaries.

```
python3 main.py \  
  --analyze_attention \  
  --attention_top_n 8 \  
  --per_slice_cap 500 \  
  --max_slices_per_stain 5 \  
  --epochs 30 \  
  --load_splits /projects/e32998/STAT390_Krish/Code/Code4_reduce_runtime/data_splits/data_splits.npz  
  
echo "End time: $(date)"
```

Mandatory Task implementation

1. Remove dropout from the attention MLP: Stabilizes attention weight learning by ensuring consistent patch scoring during training.
2. Use gated attention: Enhances non-linear feature capture, allowing the model to better identify complex relationships between diagnostic patches.

```
class AttentionPool(nn.Module):
    """
    Attention pooling mechanism for MIL.
    Input: x (B, M, D)
    Output: weighted_x (B, D), optionally weights (B, M)
    """
    def __init__(self, input_dim: int, hidden_dim: int = 128):
        super().__init__()
        # add gated attention
        self.attention_V = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.Tanh()
        )
        self.attention_U = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.Sigmoid()
        )
        self.attention_weights = nn.Linear(hidden_dim, 1)

    def forward(self, x: torch.Tensor, return_weights: bool = False):
        A_V = self.attention_V(x) # (B, M, H)
        A_U = self.attention_U(x) # (B, M, H)

        gated_attention = A_V * A_U # (B, M, H)

        weights = self.attention_weights(gated_attention) # (B, M, 1)

        weights = torch.softmax(weights, dim=1)
        weighted_x = (weights * x).sum(dim=1) # (B, D)

        if return_weights:
            return weighted_x, weights.squeeze(-1)
        return weighted_x
```

Mandatory Task implementation

3. Add entropy regularization for attention:

Forces the model to broaden its "focus" across multiple patches, reducing over-reliance on single instances and improving robustness.

```
def _calculate_entropy(self, weights: torch.Tensor, epsilon: float = 1e-8) -> torch.Tensor:
    """
    Task 3  $H(w) = -\sum(p * \log(p + \epsilon))$ 
    """
    return -torch.sum(weights * torch.log(weights + epsilon), dim=-1).mean()

def _get_all_entropies(self, attn: Dict[str, Any]) -> torch.Tensor:
    """
    Case -> Stain -> Slice -> Patch
    """
    entropies = []

    # 1. Case-level
    if "case_weights" in attn:
        entropies.append(self._calculate_entropy(attn["case_weights"]))

    # 2. Stain & Slice-level
    for _, d in attn.get("stain_weights", {}).items():
        if "slice_weights" in d:
            entropies.append(self._calculate_entropy(d["slice_weights"]))

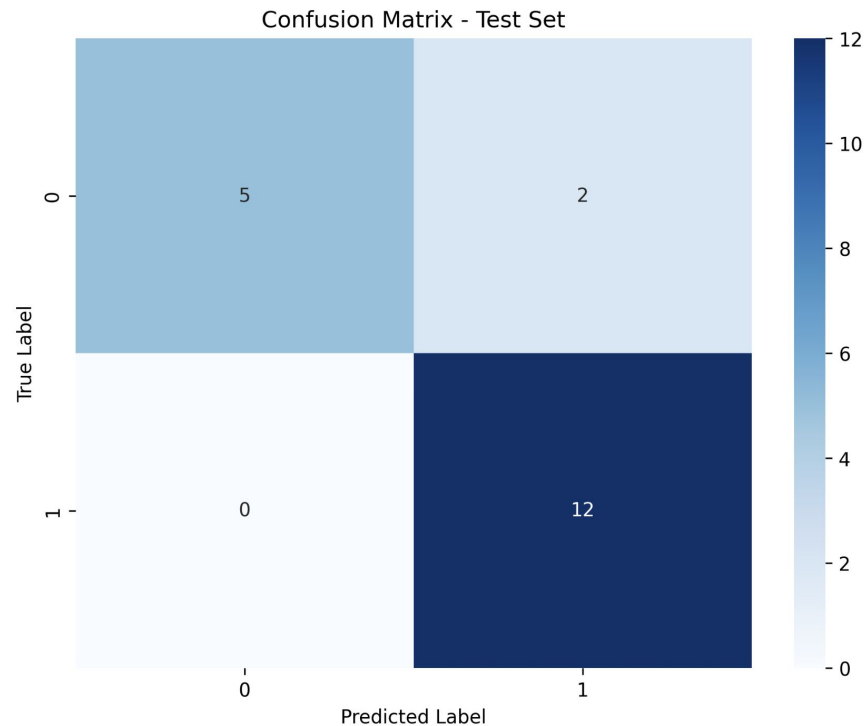
    # 3. Patch-level
    for pw in d.get("patch_weights", []):
        entropies.append(self._calculate_entropy(pw))

    return torch.stack(entropies).mean() if entropies else torch.tensor(0.0, device=self.device)
```

Result

While maintaining 100% Malignant Recall, our implementation of Gated Attention and Entropy Regularization focused on reducing false-positive triggers in benign cases. We achieved a 71.4%

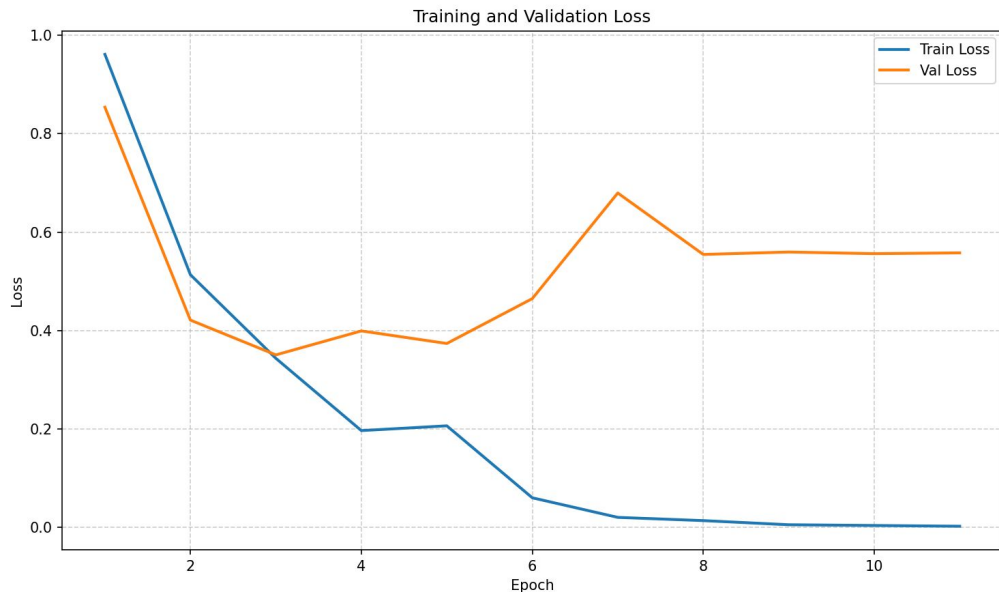
Benign Recall on the test set, successfully correctly classifying 5 out of 7 challenging benign samples.



Visualization & Analysis

Zijin Zeng

Training and validation loss



Epoch 11/30 (LR: 4.45e-05)

Train Loss: 0.0022 | Val Loss: 0.5579 | Val Acc: 0.8947

Epoch Time: 698.83s

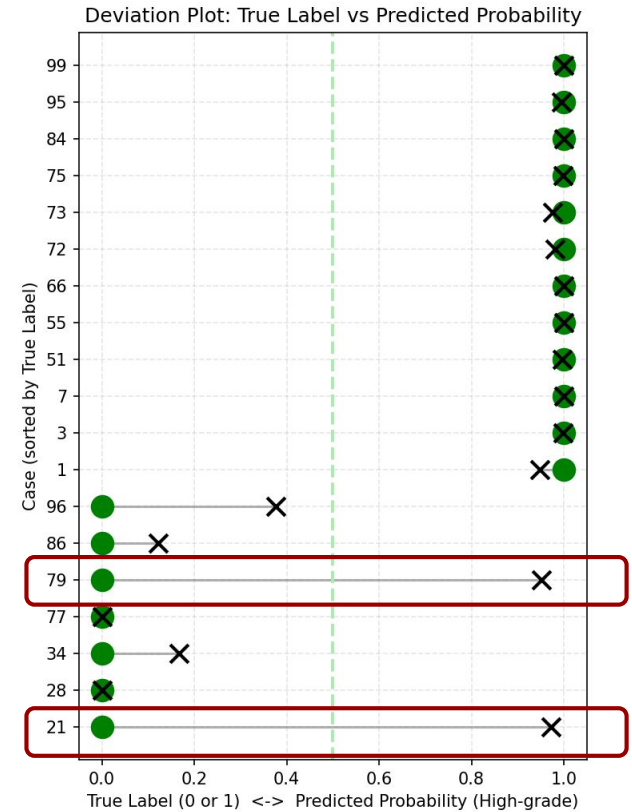
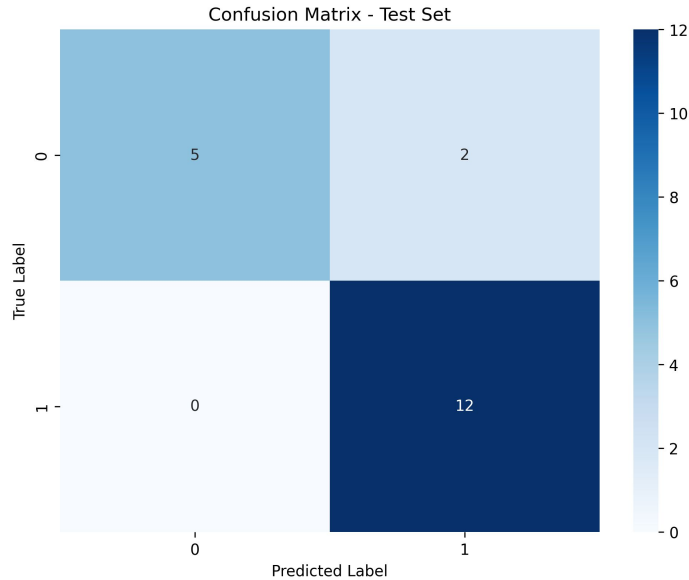
No improvement for 8 epoch(s)

Early stopping

```
'early_stopping': True,  
'early_stopping_patience': 8, # Sto  
'early_stopping_min_delta': 0.001,  
'early_stopping_min_epochs': 10 # M
```

https://github.com/arvindkrishna87/STAT390_Winter2026/blob/master/Team6/Code/Code3_NewBaseCode_3Mar_Team6/presentation3_visualization.ipynb

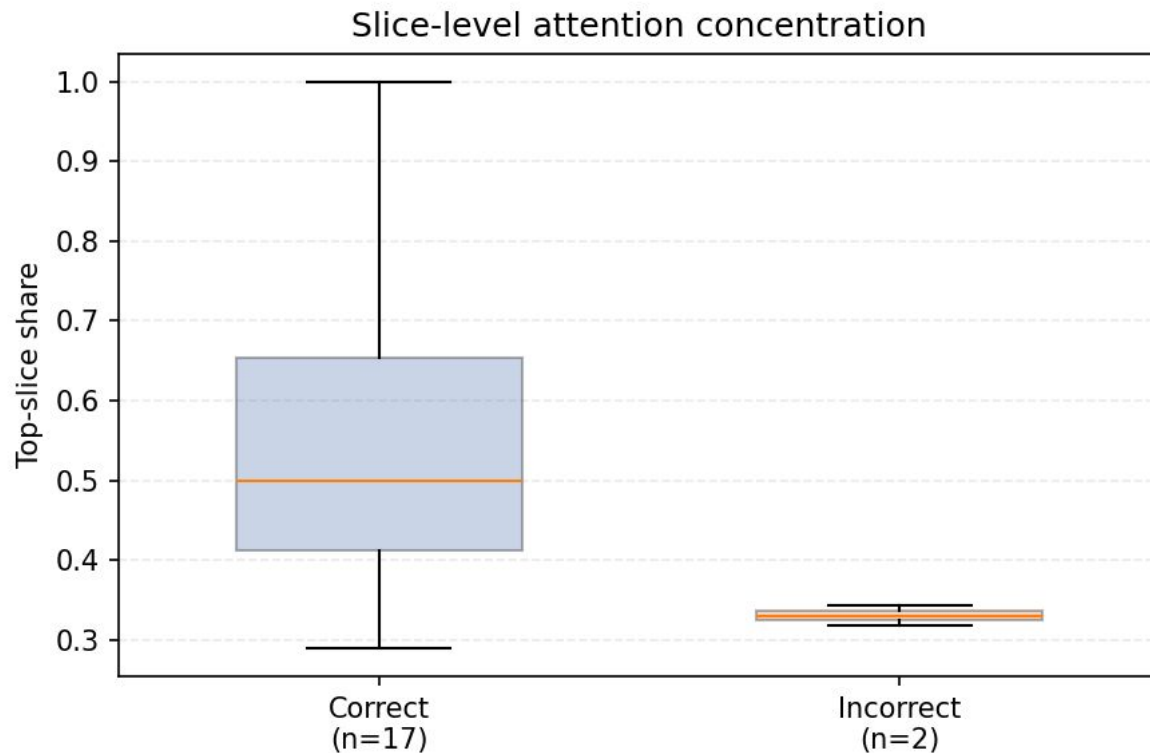
Confusion matrix & Deviation Plot



https://github.com/arvindkrishna87/STAT390_Winter2026/blob/master/Team6/Code/Code3_NewBaseCode_3Mar_Team6/run_20260303_144900/confusion_matrix.png

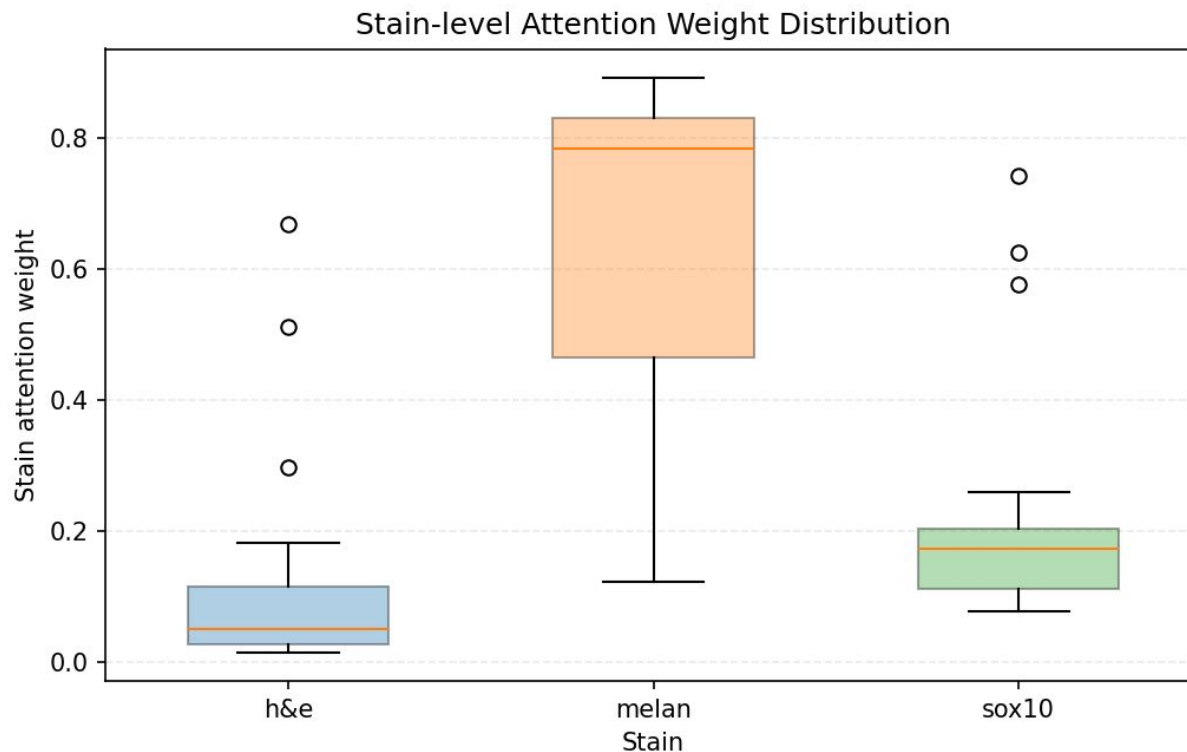
https://github.com/arvindkrishna87/STAT390_Winter2026/blob/master/Team6/Code/Code3_NewBaseCode_3Mar_Team6/presentation3_visualization.ipynb

Slice-Level Attention Concentration by Classification Accuracy



https://github.com/arvindkrishna87/STAT390_Winter2026/blob/master/Team6/Code/Code3_NewBaseCode_3Mar_Team6/presentation3_visualization.ipynb

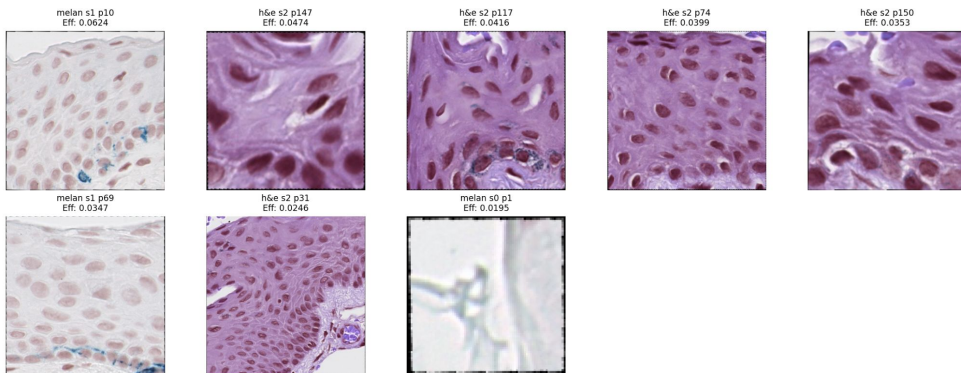
Stain—level Attention Weight Distribution



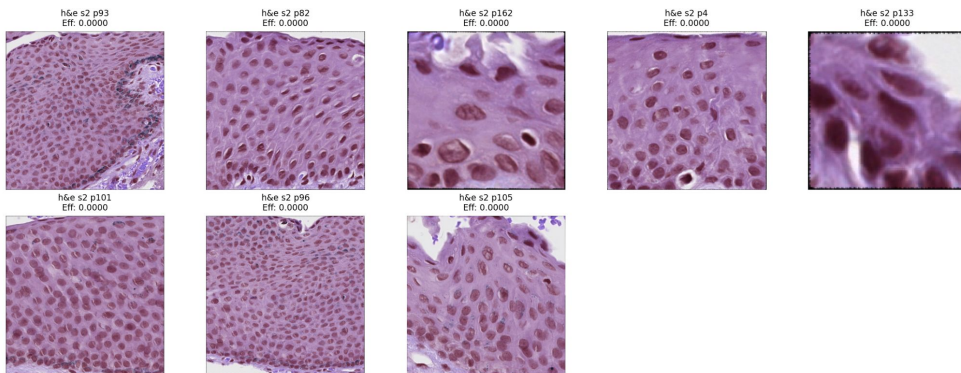
https://github.com/arvindkrishna87/STAT390_Winter2026/blob/master/Team6/Code/Code3_NewBaseCode_3Mar_Team6/presentation3_visualization.ipynb

Effective Patches For Misclassified Cases

Case 21 - Top Effective Patches



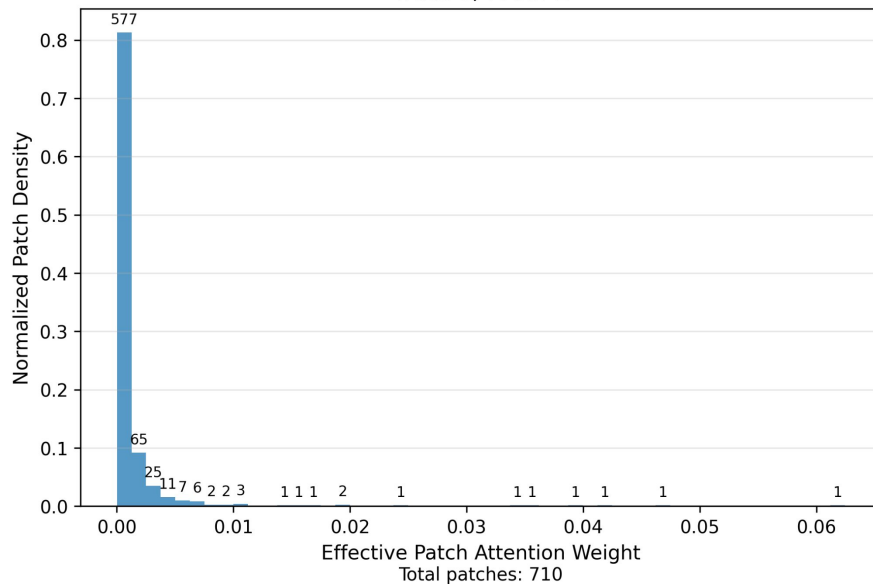
Case 21 - Bottom Effective Patches



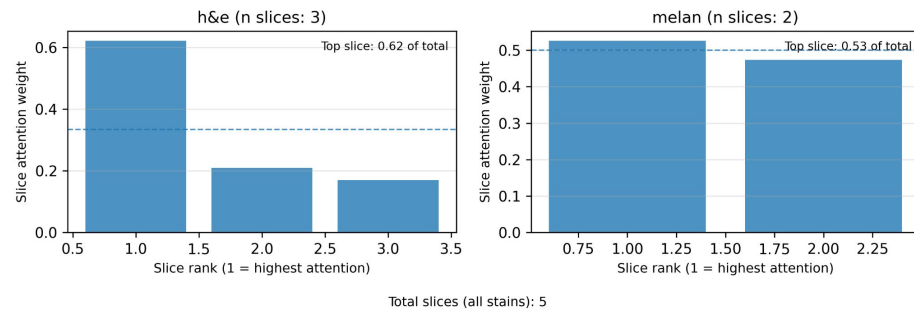
https://github.com/arvindkrishna87/STAT390_Winter2026/tree/master/Team6/Code/Code3_NewBaseCode_3Mar_Team6/run_20260303_144900/attention_analysis

Effective Patches For Misclassified Cases

Case 21 - Effective Patch Attention
True: 0, Pred: 1



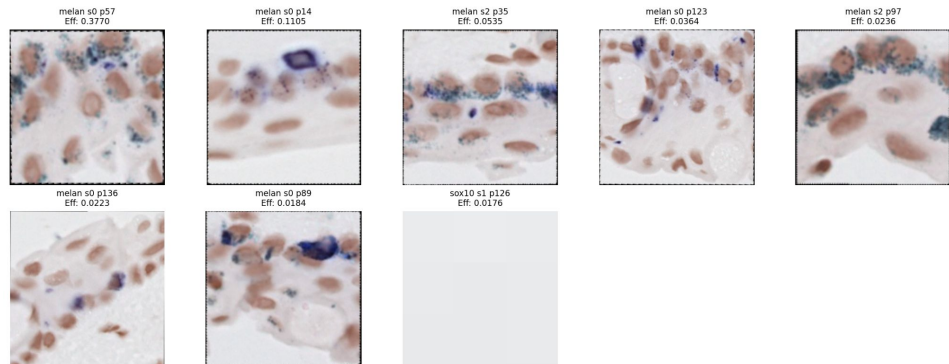
Case 21 - Slice-Level Attention by Stain
True: 0, Pred: 1



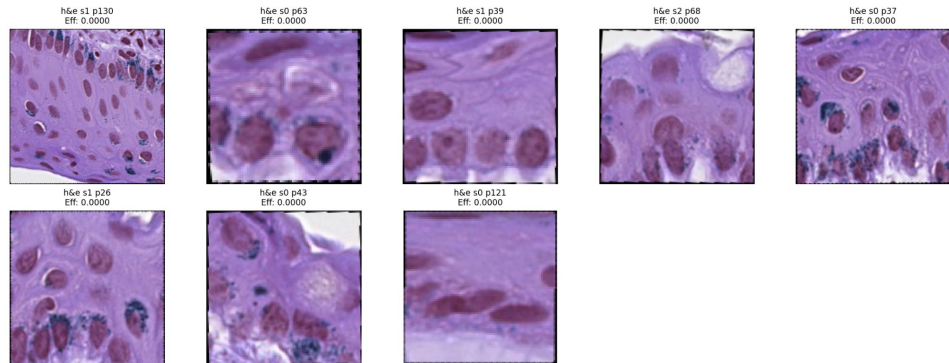
https://github.com/arvindkrishna87/STAT390_Winter2026/tree/master/Team6/Code/Code3_NewBaseCode_3Mar_Team6/run_20260303_144900/attention_analysis

Effective Patches For Misclassified Cases

Case 79 - Top Effective Patches

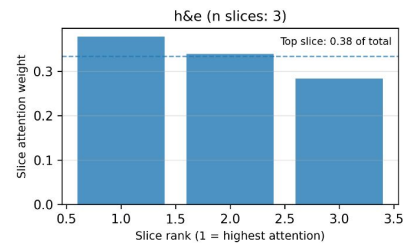
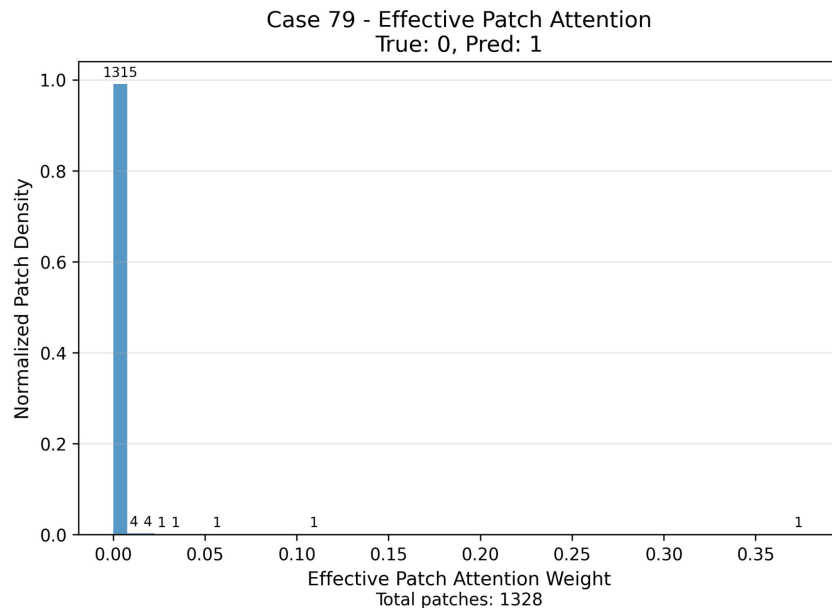


Case 79 - Bottom Effective Patches

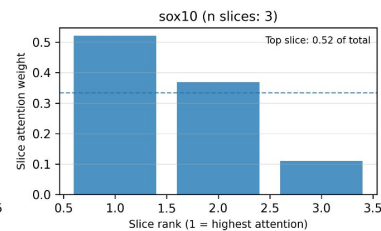
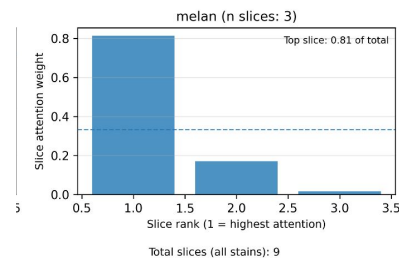


https://github.com/arvindkrishna87/STAT390_Winter2026/tree/master/Team6/Code/Code3_NewBaseCode_3Mar_Team6/run_20260303_144900/attention_analysis

Effective Patches For Misclassified Cases



Case 79 - Slice-Level Attention by Stain
True: 0, Pred: 1



https://github.com/arvindkrishna87/STAT390_Winter2026/tree/master/Team6/Code/Code3_NewBaseCode_3Mar_Team6/run_20260303_144900/attention_analysis

Visualize the high attention patches

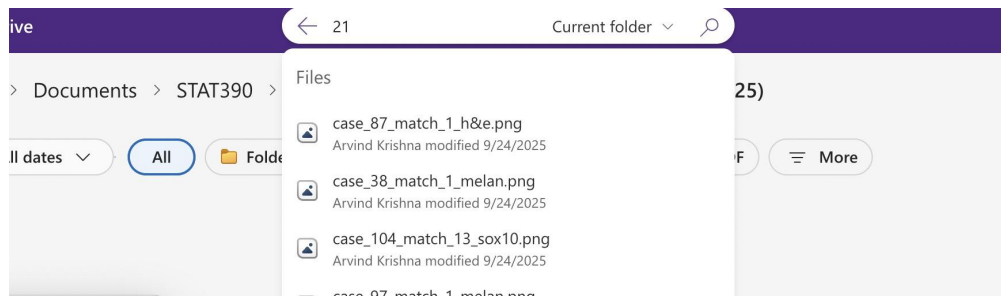
To analyze **where informative regions occur within each histopathology slice.**

- **Boundary-guided placement:** Patches are generated along the tissue boundary to capture regions likely relevant for classification.
- **Adaptive patch size:** Patch size is determined by the local tissue thickness to reduce background and preserve relevant structures.
- **Overlap control:** Overlap filtering prevents redundant patches covering similar regions.
- **Visualization:** Patches are overlaid on the original slice and colored by their weights to analyze their spatial distribution and importance.

https://github.com/arvindkrishna87/STAT390_Winter2026/blob/master/Team6/Code/Code3_NewBaseCode_3Mar_Team6/Patching_Code_updated.ipynb

Visualize the high attention patches

- mask (.png) → find contour → generate patch positions → apply positions on original image (.tif)
- Requires all .png masks & .tif slices for tracing (at least for misclassified cases)
- Inconsistent file structure and naming
- Missing .png masks



https://github.com/arvindkrishna87/STAT390_Winter2026/blob/master/Team6/Code/Code3_NewBaseCode_3Mar_Team6/Patching_Code_updated.ipynb